

X3D-S Wrist-Crop Fusion: Macro Plan

Context

The BST plateau is signal-bound on the smash↔wrist_smash pair. Pose-2D throws away the racket-pixel motion that distinguishes a wrist-flick from a full-arm smash, and the train-test gap concentrates on this pair (14-18 pp) while pose-distinctive classes generalise to within 1-2 pp at 0.95+ test F1. X3D-S on a wrist crop is the planned signal-side intervention. Capacity-side and loss-side levers have already been mapped; the wipe-drop fix lifted both pair members for the first time without a trade-off, but the residual gap is still where this branch lives.

This document is the *macro* roadmap. Each numbered stage below becomes its own research / planning / refinement / implementation task, in order. The macro's job is to surface every question that needs answering before each stage can be locked, not to answer them. Where existing code or research already covers part of a question, that gets flagged so the per-stage task starts from the right entry point rather than re-litigating settled ground.

Scope is the X3D-S branch from data derivation through fused training. Out of scope: BST-side capacity Run 2 (already queued), the augmentation set landing (queued), Phase 3 amateur-generalisation work, and the April 9 prior research doc (`architecture_1_bst_3dcnn_racket_extension_09_April.md`), which is superseded by `arch_1_directions.md:124-150` on model choice and input shape but is still useful as a citation anchor for fusion-method literature.

Anchor reference points (already settled or partial)

These are the parts the macro plan does not re-open. Each carries a source pointer for the per-stage tasks.

- Model: X3D-S, K400 pretrained via `torch.hub.load('facebookresearch/pytorchvideo', 'x3d_s', pretrained=True)`. Selected against XS / M / L / XL. SSV2 weights not officially available for X3D, parked. Source: `arch_1_directions.md:124-131`.
- Target input shape: 39 frames × stride=1, fine-tuning toward this from K400's default 13 × stride=6. RF reaches the full window by the final conv block; ~40 frames is the upper limit. Source: `arch_1_directions.md:133-135`.
- Hit-frame derivation: Method A (CSV correlation, deterministic) + Method B (shuttle horizontal-velocity sign reversal, independent verification). Re-extraction of source clips not required. Method A scaffold already exists at `src/bst_refactor/validation_scripts/hit_frame_lookup.py`. Source: `augmentation_framework.md:790-868`.
- Existing data path:
 - Per-clip .mp4 in `BST_CLIPS_DIR` (`/scratch/comp320a/ShuttleSet/clips/{split}/{Top|Bottom}_{stroke}/*.mp4`); 1920×1080 H.264.
 - Per-clip MMPose npy at `BST_MMPPOSE_NPY_DIR/{stem}_joints.npy` (F, 2, 17, 2), `_pos.npy`, `_failed.npy`. Wrists are COCO indices 9 (left) / 10 (right); torso anchors are shoulders 5/6 and hips 11/12.
 - Collated tensors at `npz_wipe_drop/{train,val,test}/`: pose, pos, shuttle, videos_len, labels, all padded to `seq_len=100`.
 - `clips_master.csv` carries `clip_stem`, `raw_type_en`, `player_side`, split column, and the rally/ball_round indices needed to rederive the windowing.
- Keypoint loss rate: 0.93% either-slot post-sticky_anchor; 6.25% shuttle-only. Per-class hit-zone fail rate cut to 0.58%. Source: `frame_zeroing.md`.
- Active baseline against which X3D-S fusion will be A/B'd: `run_20260503_172922` (combo A nosides + LS=0.0 + CDB-F1{tau=1, gamma=1, momentum=0.9, warm_up_epochs=5, f1_floor=0} + wipe_drop), mean macro 0.7481 / min ws 0.4742 / acc 0.7653 / top-2 0.9353. Best S2 0.7559 / 0.4935.
- BST fusion-injection points already mapped: post-temporal CLS (3 streams × `d_model=100`), post-interactional CLS (per-player conclusion), pre-head concat (`bst.py:415`, `(b, 300)` → head). Source: BST architecture audit.

Stage 0 — Pre-flight gates

Things that aren't a stage of their own but must land before any stage runs.

- **Code dedup (already done):** `bst_common.py` exists at `src/bst_refactor/stroke_classification/main_on_shuttleaset/bst_common.py` carrying `MODELS`, `Tee`, `build_bst_network`, `derive_active_classes_from_labels`, and `compute_data_provenance`. The X3D-S training script imports from there, no further extraction needed; the `arch_1_directions.md:472-473` "leave it for now" note pre-dates this refactor.
- **Active-class wiring:** any new training script must source head dim from `task.n_active_classes` and run `_validate_and_record_arch` on serial 1 per `arch_1_directions.md:150`. Hardcoding `taxonomy.n_classes` would put the unknown ghost back. `bst_common.derive_active_classes_from_labels` is the helper.
- **Storage location:** artefacts go on /scratch on engelbart (extraction host), rsync'd to bourbaki post-extract per the existing cross-node pattern. Local disk is not a candidate.
- **Solo X3D-S baseline gate:** before fusion, X3D-S is trained alone on the wrist crops as a 14-class classifier. The solo number is the lower bound for "fusion adds something". This sits inside Stage 5 but must be planned for in Stage 4's storage layout (the same artefacts feed solo and fused).
- **Capacity Run 2 + augmentation landing:** scheduled before X3D-S fusion build per the active-priorities ordering at `arch_1_directions.md:51-56`. The fusion baseline is whatever's best after those land, not `run_20260503_172922` directly.

Stage 1 — Hit-frame metadata derivation

Goal: produce a sidecar `hit_frame_idx.npy` per split (train/val/test), aligned to the existing collated tensors, plus a diagnostic comparison between Method A and Method B.

Existing entry point: `src/bst_refactor/validation_scripts/hit_frame_lookup.py` already implements Method A's CSV-correlation logic (per-clip → o-based hit-frame index in the clip on disk). It is library-shaped, not yet wired to a sidecar writer.

Working hypothesis (revised): Method B (shuttle-trajectory inversion) may end up being the primary, not just verification. Reasoning: ShuttleSet's CSV annotations carry visible jitter (clips_master rows show this on inspection of repeated stems), and the windowing-rule reconstruction inherits whatever drift the original annotators introduced when they marked the contact frame by eye, especially on fast strokes. TrackNet's velocity-sign reversal is a direct physical signal: the bird's horizontal velocity flips at contact regardless of who hit it. The variance comparison Method A vs Method B is therefore a primary diagnostic, not just a quality check on Method A. Whichever shows tighter per-stroke variance against a hand-checked sample becomes the canonical hit-frame source.

Open questions:

1. **Method A and Method B variance comparison.** Hand-check N=20-30 clips per stroke against the source video; report per-stroke variance for each method (mean absolute disagreement against hand-truth, with the spread). Whichever method has lower variance becomes primary; the other becomes the verification flag. Disagreement threshold for clip suspicion (1 frame? 3? 5?) and what happens to flagged clips (exclude, anchor on the primary, carry a `widen_window` confidence flag the X3D-S branch can use) follow from the variance numbers rather than being decided up-front.
2. **Method B implementation choices.** Pre-smoothing window (3-5 frame moving average per the lit-anchor); reversal-detection robustness on shuttle streams with TrackNet inpaint; handling the 1-reversal / 2-reversal / 3-reversal cases per `augmentation_framework.md:822-831`. Test-suite cases listed there are a starting set, not exhaustive.
3. **Windowing-offset alignment.** `hit_frame_lookup.py` returns the hit index *within the clip on disk*. The collated tensor is padded to `seq_len=100` with `videos_len[i]` real frames; the disk clip and collated window may not be identical lengths if the clip was shorter than 100 (stays as-is, leftover zero-padded) or windowed (truncated). Need to verify: is the collated tensor's frame 0 always the disk clip's frame 0? Or is the collation logic aligning to hit-frame? If aligned, the hit index in the collated tensor differs from the hit index in the disk clip and Stage 4 needs the disk-clip index, while a fused dataloader needs the collated-tensor index.

4. **Edge clips.** Clips where the hit lands within ± 19 frames of the disk-clip start or end. Frequency? If common, padding strategy decided in Stage 4 cascades through whether Stage 1 also flags them.
5. **Sidecar layout.** One npy per split aligned to collated tensors (`(n_clips, ) int`)? A second npy carrying confidence/method-source? A small CSV with stem $\rightarrow$ method-A-idx, method-B-idx, agreement-flag for QA? CSV is cheap and human-readable for the diagnostic pass.
6. **Test gating.** What's the manual-spot-check sample size that gates landing? Random N clips per stroke class, hand-checked against video?
7. **Validation harness.** Should this stage emit a graph/report (per-class hit-frame distribution, A-vs-B disagreement histogram, clip-position distribution of the hit) similar to existing `zeroed_frames_analysis_outputs/` ?

Deliverable shape: `validation_scripts/hit_frame_derive.py` (extends existing module), sidecar `hit_frame_idx.npy` + diagnostic CSV per split, a markdown report under `scratch/architecture_notes/` summarising A-vs-B agreement.

Dependencies: none. Self-contained on existing CSV + collated trees. Stages 2-4 all depend on this output.

Stage 2 — Wrist-keypoint loss assessment and interpolation viability

Goal: empirically decide whether wrist-keypoint interpolation is worth building, by quantifying loss rate specifically in the X3D-S window (± 19 frames around hit) for the dominant wrist of the labelled player.

Scope note — wrist keypoints, not shuttle: keep this distinct from the shuttle-loss story. Shuttle loss concentrates in high-arc classes (long_service 24.7%, smash 13.7%, clear 11.9%, lob 9.1%, ws 8.5%, drop 8.1% per

`validation_scripts/zeroed_frames_analysis_outputs/perclass_clip_miss_rate_engelbart_2026-05-03_1136.md`) and is mostly the bird leaving the top of the broadcast frame on high arcs (61.6% of gap boundaries in the top 10% per `shuttle_gap_y_distribution.py`); it's a location-conditional sensor limit on the shuttle stream, not a wrist-keypoint problem. The X3D-S branch ingests pixels, not the shuttle .npy, so the shuttle gap distribution is irrelevant here. What Stage 2 measures is the MMPose wrist-keypoint loss rate at the labelled player's dominant wrist within the ± 19 -frame hit window.

User intuition: ~1% overall keypoint loss; if that holds in the hit window for the dominant wrist, interpolation may be a sub-1pp lever and not worth the engineering. Needs validation rather than assumption — pose-fail rates aren't uniformly distributed across frames (sticky_anchor cut overall to 0.93%, hit-zone to 0.58% per

`mmpose_heuristic/phase1_vs_phase2_2026-04-29.md`), and the per-joint wrist rate inside the hit window has not been broken out yet.

Open questions:

1. **What's the actual rate?** Per-class wrist-keypoint loss rate within the ± 19 -frame hit window, for the labelled player's dominant wrist (uses Stage 1 + a Stage-3-preview dominant-wrist guess). This is the headline number that decides the rest of Stage 2.
2. **Distribution over frame-distance from hit.** Does loss concentrate at the hit frame itself (where the wrist is moving fastest) or scatter? A loss-rate-vs-frame-distance histogram answers it.
3. **Per-class pattern.** Smash and wrist_smash are the bottleneck pair; if they have higher wrist-loss rates than e.g. drives or services, that's load-bearing for whether interpolation is bottleneck-relevant or just floor-noise.
4. **Burst structure.** Are losses isolated single frames (cheap to fill with linear interp from neighbours) or runs of 5-10 consecutive frames (much harder)? Single-frame losses are mostly recoverable; multi-frame runs may need temporal smoothing or carry-last.
5. **Interpolation method shortlist** (only relevant if rate > some threshold):
 - Linear between bookend valid frames (cheapest).
 - One Euro Filter for adaptive low-pass on the wrist trajectory (recommended at `architecture_1..._09_April.md:95` for stroke-velocity-aware smoothing).
 - Carry-forward / EMA fallback for unbookended runs at clip boundaries.
 - Or no interpolation, accept the loss.

6. **Where does interpolation live?** A pre-extraction step that fixes the joints.npy in-place, a per-batch step in the dataloader, or a per-clip preprocessing step at extraction time (Stage 4)? Three plausible homes; choice depends on what the X3D-S branch needs.
7. **Flag-keep vs. flag-mask.** If we interpolate, does the model get a `wrist_was_interpolated` flag channel or does the interpolation flow through silently? The shuttle-mask variant 2a result (`run_20260503_192718`), no benefit on top of unzeroing) is a relevant precedent: the model may be inferring missing-frame structure from context already.

Deliverable shape: `validation_scripts/wrist_loss_in_hit_window.py`, output is a markdown report with text stats + matplotlib charts, gated decision on whether to build the interpolation module. If yes, a separate Stage-2.5 task spec'd from the report.

Dependencies: Stage 1 hit-frame sidecar; provisional Stage 3 dominant-wrist heuristic (or use the lazy `player_side` ground truth for the assessment pass — Stage 3.1.1's "use as test" framing).

Stage 3 — Crop sizing strategy + dominant wrist heuristic

Goal: settle the wrist-crop spec (size, anchor, smoothing, fallbacks) and the dominant-wrist picker.

3.A Crop sizing

User starting point: $\text{wrist} + 1.6 \times \text{torso_length}$ as $\pm xy$. Translates to crop side of $3.2 \times \text{torso_length}$ centred on the wrist.

Reference point for sanity: `architecture_1..._09_April.md:87-91` recommends

`crop_radius = k \times \text{torso\_height\_pixels}` with k starting at 1.0, tuned 0.8-1.2. That's a side of $2 \times \text{torso_height}$, narrower than $3.2 \times \text{torso}$. Forearm + racket together span ~93cm vs 50-55cm torso, so the racket-head-inclusive radius needs to be at least $\sim 1.0 \times \text{torso}$, i.e. $\text{side} \geq 2 \times \text{torso}$. Anything wider trades crop content density for safety margin against mis-anchor.

Provisional recommendation: lead with $k = 1.0-1.2$ (crop side $2.0-2.4 \times \text{torso}$), not 1.6 (side $3.2 \times \text{torso}$). Reasoning: at 112^2 output resolution, a $3.2 \times \text{torso}$ crop puts the racket-and-wrist content into ~30-40% of the frame and the rest is contextual body / court padding. At $2.0 \times \text{torso}$ the racket and wrist fill ~60-70% of the frame, which is what the X3D-S kernels are sized to discriminate. The wider crop adds safety margin against mis-anchor (helpful if Stage 1 hit-frame slips by a few frames or the wrist keypoint drifts) but the X3D-S branch isn't where you want to be paying for safety margin: Stage 1 + Stage 3.B should be tight enough that the crop centre is reliable. Concrete proposal for Stage 3: visual-inspection pass on $N=20$ clips per stroke at $k \in \{0.8, 1.0, 1.2, 1.6\}$; lock the smallest k where the racket head sits inside the crop on $\geq 95\%$ of inspected frames. Default lock if the inspection comes back inconclusive: $k = 1.2$.

Open questions:

1. **$k = 0.8, 1.0, 1.2$, or 1.6 ?** User's $1.6 \times$ as $\pm xy$ is wider than the literature anchor of $1.0 \times$ as radius (which gives $k=1.0$ in their formula). Reconcile against what's in the crop at each k : does the racket head + grip + wrist + a ~10% margin fit? Try a visual-inspection pass on $N=20$ clips per stroke at $k \in \{0.8, 1.0, 1.2, 1.6\}$.
2. **Square or rectangle?** April 9 doc argues square because racket can point any direction. Agreed unless a per-class direction prior changes the calculation.
3. **Centre on wrist, no racket-head offset.** April 9 doc suggested offsetting $0.3 \times \text{torso}$ along the elbow→wrist direction to capture the racket head beyond the wrist. Dropped: the racket isn't a constant extension of the forearm — grip rotation, pre-cock, follow-through, and around-the-head shots all break the assumption. The crop has to be wide enough on its own to cover the racket head in any direction the wrist is pointing, which is what the k sizing question above is for. Centre on the dominant wrist; let the crop side carry the safety margin.
4. **Torso_length measurement.** Mid(shoulder_5, shoulder_6) to mid(hip_11, hip_12)? Or the longer of shoulder-to-hip per side? Or shoulder-width $\times 1.5$ fallback when hip keypoints fail?
5. **Fallback hierarchy when keypoints missing.** April 9 doc proposed: torso $\rightarrow$ shoulder-width $\times 1.5 \rightarrow$ upper arm $\times$ constant $\rightarrow$ bbox-height $\times 0.3$. Agree with this ladder? What's the rate at which each tier kicks in?
6. **Temporal smoothing of crop centre and radius.** Raw keypoints jitter 3-10 px frame-to-frame. One Euro Filter is the lit anchor; $\text{min_cutoff} \approx 1.0$ Hz, $\text{beta} \approx 0.5$ for centre, $\text{beta} \approx 0.1$ for radius. Worth adopting? Or is per-frame raw acceptable since X3D-S has temporal conv that can absorb jitter?

7. **Edge handling.** Wrist near broadcast edge → crop falls partly outside the frame. Options: shift crop centre inward (deforms the crop content), zero-pad the off-frame region (most common), reflect-pad (smooth, natural). Choose one default with rationale.

3.B Dominant wrist heuristic

User starting point: absolute magnitude of wrist displacement across the clip; wrist with larger displacement = dominant wrist.

User's framing on validation: lazy option is the `player_side` + per-clip dominant-hand metadata (if present); preferred path is a heuristic that doesn't require it, with the metadata kept as ground truth for QA.

Provisional recommendation: lead with hit-window peak velocity rather than whole-clip absolute displacement. Reasoning: whole-clip displacement integrates over walking, ready-stance shifts, and recovery motion, all of which are roughly symmetric across both wrists for most strokes; the asymmetry that identifies the strike wrist concentrates in the few frames around contact. Peak velocity within ± 5 frames of the Stage-1 hit frame isolates the same signal at higher SNR. Whole-clip absolute displacement remains a sensible fallback when the hit-window keypoints are zeroed (sub-1% case per Stage 2 stats). Operationally: rank wrists on `max(|d/dt wrist|)` over `hit  $\pm$  5` frames, fall back to whole-clip path length if both wrists are missing in the hit window, fall back to shuttle-distance-at-hit if displacement is uninformative on both wrists. Validation rubric stands as written below.

Open questions:

1. **Displacement window scope.** Whole clip vs ± 19 frames around hit vs ± 5 frames around hit? Whole-clip displacement is most robust but contaminated by non-stroke motion (the player walking around). Hit-window displacement is cleaner but smaller sample.
2. **Displacement metric.** Total path length (sum of frame-to-frame distances), peak velocity, peak acceleration, peak jerk? Each isolates a different stroke characteristic.
3. **Relative to shuttle.** At the hit frame, the wrist closer to the shuttle is by definition the strike wrist. Combine with displacement as a tie-breaker, or use as the primary signal.
4. **Robustness on missing-keypoint clips.** If the dominant wrist is the one that's intermittently zeroed, displacement metrics can flip sign. Test on the bottom-quartile clips by wrist-keypoint loss rate.
5. **Ambidextrous / non-dominant edge cases.** Around-the-head shots use the dominant wrist on the wrong side of the body. Backhand strokes use the dominant wrist but with a different swing path. Flag these as known failure modes; assess heuristic agreement on the `aroundhead` and `backhand` filtered subsets in `clips_master.csv`.
6. **Per-clip vs per-frame.** Same wrist for the whole clip (standard), or allow switching mid-clip (probably never needed, but flag). One choice for the whole pipeline.
7. **Validation rubric.** Use ShuttleSet's own dominant-hand metadata (if accessible per player) as ground truth on a stratified sample, report agreement rate, hand-check the residuals. Heuristic ships if agreement $\geq \sim 98\%$ on non-edge cases.

Deliverable shape: a `wrist_crop_spec.py` module exposing `pick_dominant_wrist_idx(joints, hit_frame_idx) → 9 | 10` and `crop_box(joints, frame_idx, dominant_idx) → (x1, y1, x2, y2)` with the chosen sizing + smoothing + fallbacks; a validation script that reports dominant-wrist agreement against metadata; markdown writeup under `scratch/architecture_notes/`.

Dependencies: Stage 1 hit-frame; Stage 2 may flag clips that need wrist interpolation before the heuristic runs.

Stage 4 — Wrist-crop extraction pipeline

Goal: extract per-clip wrist-crop video tensors at X3D-S input shape, frame-exact, padded as needed, for the full 32k-clip set.

Constraint: the source-clip extract took ~ 2 days; we cannot afford that timescale again. Per-clip cropping from the existing per-clip .mp4 is dramatically cheaper because the work per clip is reading + slicing + writing a much smaller artefact, no re-decoding of source match videos. Ballpark expectation: 32k clips on a single GPU with multi-process

readers = hours, not days.

4.A Extraction mechanics

Open questions:

1. **Read path.** PyAV (Python bindings to ffmpeg, fast random-access decode) vs cv2 (familiar but slower) vs torchvision.io.read_video (built-in but limited). PyAV is the standard for frame-exact decoding of subsets.
2. **Resolution to extract at.** Native X3D-S pretraining is 160×160 (per April 9 doc); the same doc argues 112×112 to drop GFLOPs from 1.96 to 0.96 with negligible perf loss; the user has flagged “max input usable by X3D-S” as the question. Likely settle at 112×112 or 160×160; pick before Stage 4 ships because storage size and training cost both scale on H².
3. **Codec — recommend raw uint8 stacked .npy.** H.264 at crf ~18 saves ~70-80% on disk (~3-5 GB at 112², ~7-10 GB at 160²) but pays per-clip decode cost every epoch on every batch — typically 5-30ms per clip depending on GoP structure, which on a 128-batch dataloader compounds into a real bottleneck unless aggressively cached. Raw uint8 stacked per split (~17 GB at 112², ~35 GB at 160²) is memory-mappable from /scratch, gives constant-time random access, and removes decode entirely from the train loop. Storage budget on /scratch easily absorbs either; user has confirmed the extra storage is fine if it speeds training. Lock raw uint8 .npy unless the resolution decision pushes the artefact past ~50 GB. Stacked per-split mirrors the existing pose/pos/shuttle pattern and slots into the same dataloader idiom.
4. **Frame-exact guarantee.** Use PyAV with explicit PTS-based seeking, not `-ss` keyframe-only seek. Test against a hand-checked sample (sync extracted frame N against the source-clip frame N visually).
5. **Resolution downsample method.** cv2.INTER_AREA (best for downsampling), bilinear, lanczos. INTER_AREA is the standard.
6. **Channel order.** RGB or BGR. PyTorchVideo’s X3D-S expects RGB normalised with mean/std per the ImageNet/Kinetics convention; lock this and test the first batch against the model’s expected stats before training.
7. **Parallelism.** Per-clip extract is embarrassingly parallel; a multi-process pool with N workers (where N matches CPU cores, capped by IO contention) over 32k clips. What’s the achievable throughput on engelbart vs bourbaki?

4.B Padding for short windows

Open questions:

1. **Hit too close to start/end.** Clips where the hit is within <19 frames of disk-clip start or end. Frequency comes from Stage 1’s diagnostic. Padding options:
 - Zero-pad the missing frames (model sees black on one side; safe but throws away information).
 - Replicate the boundary frame (model sees a frozen image; introduces no-motion artefact).
 - Truncate the X3D-S window asymmetrically and zero-pad to 39 (reduces effective context).
 - Skip the clip from X3D-S training and let the fusion fall back to BST-only on those clips.
2. **Spatial padding for off-frame crops.** When the wrist crop falls off the broadcast edge: zero-pad (black, standard), reflect-pad (smooth), replicate (extends edge). Zero-pad is the standard for video models pre-trained on Kinetics-style content where black borders are natural.
3. **Padding mask.** Pass a per-frame `is_real` mask to X3D-S? Or accept the zero-frame artefact silently? X3D-S has no native mask input; the closest is letting the model learn to ignore black frames as a low-signal pattern.

4.C Storage and dataloader integration

Open questions:

1. **Per-clip vs stacked.** Per-clip .mp4 keeps each artefact small and lazy-loadable; stacked-per-split .npy is faster at training time but inflexible to schema changes. Stacked is the existing pattern for joints/pos/shuttle.
2. **Sync with the existing dataloader.** The dataset class at `shuttleset_dataset.py:139-238` returns `((human_pose, pos, shuttle), videos_len, labels)`. The X3D-S branch adds `wrist_crop` as a fourth array. Either extend the existing dataset or build a parallel one and zip them.

3. **Memory layout at training time.** 17 GB raw at 112^2 fits comfortably as a memory-mapped .npz on /scratch with no preload required (np.load(..., mmap_mode='r') gives random-access at OS-page granularity); 35 GB at 160^2 same story. CPU RAM on the training nodes can preload the whole training split if desired but is not required.
4. **Cross-node propagation.** Extract on engelbart, rsync the stacked .npz to bourbaki post-extract per the existing cross-node convention (matches Phase 2 collation). Single rsync pass per split per resolution; storage on both nodes' /scratch.

Deliverable shape: pipeline/wrist_crop_extractor.py (writes the crop artefacts), preparing_data/wrist_crop_dataset.py (loads them), an extract validation script + log, the artefact tree under BST_WRIST_CROP_DIR (env var, mirrors the existing pattern).

Dependencies: Stages 1, 2, 3 all feed in.

Stage 5 — X3D-S solo fine-tune

Goal: ship X3D-S as an independent stroke classifier on the wrist crops, on the same active taxonomy/split (combo A nosides) as BST. The solo number is the lower-bound check (“does the wrist-crop signal carry stroke information at all?”) and a candidate-warm-start for fusion.

5.A Stage progression

User’s question: K400 default ($13 \text{ frames} \times \text{stride}=6$) → target ($39 \times \text{stride}=1$) is a big jump. Stage progression options.

Options:

1. **Direct jump.** K400 weights → $39 \times \text{stride}=1$ in one fine-tune. Simplest. Risk: stride drop from 6 to 1 is dramatic and the model has never seen frame-by-frame input; pretraining on coarse temporal sampling may not transfer.
2. **Stride-staged.** K400 → $13 \times \text{stride}=6$ fine-tune on badminton → $26 \times \text{stride}=3$ fine-tune → $39 \times \text{stride}=1$ fine-tune. Each stage is a small distribution shift. Cost: $3 \times$ the training runs. Benefit: smooth transfer.
3. **Frame-staged.** K400 → $13 \times \text{stride}=1$ fine-tune (matches K400 frame count, drops stride to 1) → $39 \times \text{stride}=1$ fine-tune (extends frame count). Two stages. Tests whether stride or frame-count is the harder shift independently.
4. **Train-from-K400 directly at $39 \times \text{stride}=1$,** accept some weight transfer loss for the simpler runbook.

Need to pick one. Default recommendation absent further data: option 4 (direct jump) + a single fallback test on option 1 (the cheap step) if option 4 fails to converge. Stride and frame-count staging is overkill for the timescale unless the direct path stalls.

5.B Hparam questions

1. **Resolution.** 112^2 vs 160^2 — same question as Stage 4 but the answer here is the binding one because X3D-S’s pretrained resolution is 160^2 and dropping to 112^2 is a small additional distribution shift. Pick once for both stages.
2. **Optimizer + LR.** AdamW (matching BST) vs SGD with momentum (X3D paper standard); base LR for fine-tuning a Kinetics-pretrained backbone is typically $10 \times$ lower than from-scratch ($\sim 5e-5$ to $1e-4$) but higher for a head-only stage. Layer-wise LR decay candidate from the ViT-fine-tune literature; probably overkill for X3D-S.
3. **Batch size.** Memory cost on V100 16GB for batch_size N at $39 \times 112^2 \times 3$: a quick estimate is needed before launch. X3D-S forward is ~ 1 GFLOP at $112^2 \times 13$; at 39×112^2 it’s ~ 3 GFLOP per sample.
4. **Augmentation.** Centreline flip applies (wrist crop horizontal-mirror in lockstep with the BST flip; bilateral COCO swap is a no-op for X3D-S since the input is RGB pixels, not joint indices). Constrained jitter is a court-frame and camera-frame op; doesn’t transfer. RGB-side aug to add: colour jitter (mild brightness/contrast, court is bright neutral so saturation moves are safe), random erasing (Cutout-style). Mixup / Cutmix sit one tier higher in invasiveness; flag but probably defer.
5. **Loss.** CDB-F1 family same as BST? Or vanilla CE for the solo run to keep the X3D-S baseline interpretable? Probably CE for solo, switch to BST’s CDB-F1 once integrated for fusion.
6. **Early-stop and serial count.** Match BST’s 5-serial pattern, same early-stop on val macro F1.
7. **Pretrained-checkpoint variants.** PyTorchVideo K400 is the obvious one. Does PySlowFast carry an alternative checkpoint that performs better (different recipe)? Cross-check.

5.C What does “good” look like solo

A solo X3D-S that beats chance ($1/14 \approx 7\%$) is the floor. A solo number in the 0.55-0.65 macro F1 band is plausible given the wrist-only context (wide bands of stroke-distinctive whole-body motion are gone; only the racket-end signal remains). Anything $\geq \sim 0.5$ macro is signal-positive for fusion; below that and the wrist-crop content question reopens.

Deliverable shape: `main_on_shuttleset/x3d_s_train.py` (mirror of `bst_train.py`), checkpoint dir under `experiments/`, manifest record per the existing pattern, comparison entry in `nosides_runs_table.md`.

Dependencies: Stage 4 wrist-crop artefacts.

Stage 6 — BST + X3D-S fusion integration

Goal: the actual Arch 1 build. Two sub-questions: where to fuse (architecture), and how to schedule training (optimisation).

6.A Where/how to slot in X3D-S — research arm

Citation anchors for the Stage 6 sub-task

These are the bibliographic entry points the Stage 6 read-the-paper pass should start from. One-line summary per anchor; full paragraph reads + block quotes happen in the sub-task. Track these as a citation block in the Stage 6 writeup.

- **MMTM** — Joze, H. R. V., Shaban, A., Iuzzolino, M. L., & Koishida, K. (2020). *MMTM: Multimodal Transfer Module for CNN Fusion*. CVPR 2020. Squeeze-and-excitation cross-modal channel gating; per-stream pooled features concatenated, FC, split into per-stream channel excitations. Mid-network plug-in; preserves pretrained weights.
- **MBT** — Nagrani, A., Yang, S., Arnab, A., Jansen, A., Schmid, C., & Sun, C. (2021). *Attention Bottlenecks for Multimodal Fusion*. NeurIPS 2021. Small set of learnable bottleneck tokens mediate cross-modal attention; bounds compute on cross-modal information flow vs full cross-attention.
- **FiLM** — Perez, E., Strub, F., De Vries, H., Dumoulin, V., & Courville, A. (2018). *FiLM: Visual Reasoning with a General Conditioning Layer*. AAAI 2018. Per-channel affine (γ , β) conditioning of one stream’s features by another’s; cheap, channel-level, no spatial fusion.
- **Cross-modal cross-attention (general)** — Tsai, Y. H., et al. (2019). *Multimodal Transformer for Unaligned Multimodal Language Sequences*. ACL 2019. The natural transformer-side baseline; BST already has an internal `MultiHeadCrossAttention` that this would slot alongside.
- **Late feature concat + MLP** — standard baseline; cited via Karpathy, A., et al. (2014). *Large-Scale Video Classification with Convolutional Neural Networks*. CVPR 2014, which established the early/late fusion distinction. The pre-head concat at `bst.py:415` is the existing in-codebase analogue.
- **RacketVision (2025)** — flagged at `architecture_1..._09_April.md:52`: naive concatenation of racket pose features degraded performance vs unimodal baseline; only cross-attention fusion unlocked gains in racket-sport benchmarks. Confirm publication venue + author list as part of Stage 6.
- **X3D backbone** — Feichtenhofer, C. (2020). *X3D: Expanding Architectures for Efficient Video Recognition*. CVPR 2020. Required reading for the input-shape and stride-vs-frame-count fine-tune decisions in Stage 5 as well.
- **BST** — Chang, K., et al. (2025). *Badminton Stroke-type Transformer*. arXiv:2502.21085. The architecture being extended; predecessor TemPose is the more relevant cross-attention reference.

Existing thinking at `arch_1_directions.md:137-142` maps three shapes:

Three competing options: - **Late concat, just before the MLP head.** Easiest to implement, lowest risk, but gives BST no chance to condition its attention on the racket signal. - **Tie into attention earlier, in a meaningful way.** X3D-S output feeds into the cross-attention or the interactional transformer, so the racket evidence shapes how players and shuttle attend to each other. More expressive, more moving parts. - **Separate tower with learned significance weighting.** X3D-S runs as its own tower and a learned scalar (or vector) gates how much its prediction counts vs BST’s.

`architecture_1..._09_April.md:44-66` adds:

- **Late feature concat + 2-layer MLP** (the “Phase 1 baseline” — what `arch_1_directions.md` calls late concat).

- **MMTM (Multimodal Transfer Module, Joze et al. CVPR 2020)** — per-stream squeeze-excitation gates driven by the concat of pooled features, enabling channel-wise cross-modal modulation.
- **Bottleneck fusion tokens (MBT, NeurIPS 2021)** — 4 learnable bottleneck tokens that mediate cross-modal attention.
- **FiLM conditioning** — channel-wise affine gating from one stream onto the other.

Plus the RacketVision (2025) finding that naive concatenation degraded performance vs unimodal baseline in the racket-sport context; only cross-attention unlocked gains.

Open questions for the Stage 6 sub-task:

1. **Cite-and-read pass.** April 9 doc cites these models but as a high-level summary. The Stage 6 sub-task must read each paper of interest in full (not just the abstract or a single paragraph), with block quotes used to support the recommendation rather than to decorate it. Specifically: MMTM (Joze 2020), MBT (Nagrani 2021), RacketVision (2025), the X3D paper (Feichtenhofer 2020), TemPose (the BST predecessor, for the cross-attention context), and at least one anchor on FiLM conditioning (Perez 2018).
2. **Fusion-method shortlist.** From the literature, narrow to 2-3 candidates ranked on:
 - **Likelihood of positive lift:** how the method has fared on similar skeleton+RGB tasks, how naturally it handles the architectural heterogeneity (transformer tokens vs CNN feature maps), whether the racket-sport literature gives any direct evidence.
 - **Conceptual complexity:** how many moving parts to tune, how interpretable the fused signal is.
 - **Code complexity and bug surface:** lines added, modifications to BST internals, risk of breaking existing BST train/infer/test pipelines.
 - **Hparam search space introduced:** separate assessment per candidate (e.g. MMTM adds a bottleneck width and a temperature; MBT adds bottleneck-token count and depth-of-insertion; FiLM adds the conditioning-network depth).
3. **Fusion at what depth in BST.** Pre-temporal CLS, pre-cross-attention, pre-interactional, pre-head. The pre-head concat is the lowest-risk lift; deeper insertions raise the ceiling but introduce more places for the fusion to fail.
4. **Spatial reduction of X3D-S features.** X3D-S head outputs are clip-level (b, C). Pre-head fusion ingests this directly. Earlier-than-pre-head fusion may want temporal or per-frame X3D-S features; that requires bypassing X3D-S's avgpool and exposing a (b, C, T') feature stream. Decide what's needed before locking the fusion shape.
5. **Where the fused feature lives in the model.** New separate `Arch1` module that wraps BST + X3D-S + fusion, or BST extended with an optional `x3d_branch` flag like the existing PPF/CG/AP toggles? Latter keeps the codebase smaller; former isolates the X3D-S code.
6. **Class-head sourcing.** Single fused head over all 14 classes vs auxiliary X3D-S head (multitask loss). Multitask adds a loss-balance hparam.
7. **Inference-side handling of clips where X3D-S input is unavailable** (extreme padding cases from Stage 4). Either pad X3D-S input with zeros and let the fusion handle it, or fall back to BST-only at inference.

6.B Training schedule and hparam search

User's intuition: a few starting positions worth trying.

1. **Both pretrained, joint fine-tune from epoch 1.** Both branches at their solo-best weights, fusion module randomly initialised, train end-to-end with a low LR for both backbones.
2. **BST scratch, X3D-S pretrained.** Re-train BST from random init alongside the pretrained X3D-S, fusion learned jointly. Lets BST's representations grow with the X3D-S signal but is a much longer run.
3. **X3D-S frozen at a checkpoint before its solo optimum.** Counterintuitive but defensible: the solo X3D-S head may have specialised on cues that fusion doesn't need; an earlier checkpoint carries lower-level features that the fusion can mould to its own purpose. Trades a known signal level for plasticity.
4. **Same idea for BST.**

Open questions:

1. **Which combinations are worth running.** Cartesian over {BST-pretrained, BST-random, BST-pretrained-earlier} × {X3D-S-pretrained, X3D-S-random (probably skip), X3D-S-pretrained-earlier} × {fusion-only, end-to-end} is too many. Pick a 3-4 row matrix that brackets the main axes.
2. **Learning rate ratio between branches.** April 9 doc suggests 10× lower for the encoders during joint fine-tune; this is a sensible default but depends on the fusion-only initialisation step.
3. **Three-phase schedule.** April 9 doc proposes: (a) freeze BST and train X3D-S head; (b) freeze both encoders and train fusion only; (c) end-to-end with 10× lower LR on encoders. Question for Stage 6: does this three-phase recipe transfer to the BST architecture or is it overkill given how small the fusion module will be?
4. **Augmentation under fusion.** Centreline flip needs both branches flipped in lockstep. Constrained jitter touches BST inputs only. Are there X3D-S-specific augmentations (RGB jitter, random erasing) that need to gate on the BST flip / not desync the streams?
5. **Loss.** CDB-F1 from BST carries forward; tau and gamma might want re-tuning for the fused setup since the per-class F1 floor may shift after fusion. Or hold tau and gamma fixed initially to keep the comparison clean.
6. **Hparam search budget.** What's the realistic compute budget for Stage 6? Number of full 5-serial runs informs whether the fusion-method shortlist is N=1 (depth-first) or N=2-3 (breadth-first).
7. **Eval baseline.** A/B against `run_20260503_172922` on combo A nosides, plus the latest landed runs from Capacity Run 2 and the augmentation set. The “what does the fusion need to clear to count as a win” question wants an explicit number before launch (e.g., +1 pp macro and +2 pp wrist_smash, or whatever the user wants).

Deliverable shape: `main_on_shuttleset/arch1_train.py`, fusion module under `model/fusion/` (or extended `bst.py`), per-fusion-method ablation suite, full writeup at `scratch/architecture_notes/arch_1_directions.md`'s experiment log.

Dependencies: Stages 1-5 all upstream.

Cross-stage open questions

These cut across multiple stages and want answering early so they don't compound into Stage 6.

1. **Resolution lock.** 112² vs 160². Affects Stage 4 (storage), Stage 5 (compute), Stage 6 (compute). One number, picked once.
2. **Storage location.** /scratch on bourbaki/engelbart, with the env-var pattern (`BST_WRIST_CROP_DIR`) mirroring the existing layout. Confirm before Stage 4 launches.
3. **Code home.** Does Arch 1 live as new files under `src/bst_refactor/stroke_classification/arch1/`, or extend the existing `model/`, `preparing_data/`, `main_on_shuttleset/` trees in place? Latter is lighter-weight; former isolates the X3D-S work for cleaner branching and rollback.
4. **Splits.** Stick with combo A nosides (`une_merge_v1_nosides + split_v2 + dropunk`) for X3D-S solo and fused, mirroring active BST baseline. The cross-player swap-val-test direction (`arch_1_directions.md:443-453`) stays parked.
5. **Reproducibility scaffolding.** Run-id, manifest, per-serial seeding, weight-cache pattern, `_validate_and_record_arch` extension to record `extra.x3d_arch` (frames, stride, resolution, pretrain-source). Stage 0 cleanup work.
6. **Data versioning.** A new `ablation_id` like `arch1_v0` to mark the new collated tree variant if any of the existing tensors get re-collated to add the hit-frame sidecar or wrist-crop pointer.

Verification plan (per stage)

Each stage ships with its own validation harness. The macro plan locks the shape only.

Stage	Validation
1	Method A vs B agreement histogram per class; manual hand-check N clips per stroke against the source video; sidecar tensor unit-test (shape, dtype, no NaNs, in-bounds indices)
2	Per-class wrist-loss-rate report in the ± 19 -frame hit window; loss-distribution chart across frame-distance from hit; gated decision artefact (build interpolation Y/N + rationale)
3	Dominant-wrist heuristic agreement against <code>player_side</code> + dominant-hand metadata on stratified sample; visual-inspection grid of N=20 crops per stroke at chosen k value(s); fallback-tier-fire-rate stats
4	Frame-exact extraction validated by hash-comparing N=50 hand-picked frames decoded both via the new pipeline and via cv2/PyAV against the source clip; full-set extraction timing report; storage footprint vs estimate; spot-check decoded crops match the joint annotations
5	Standard 5-serial run on combo A nosides; entry in <code>nosides_runs_table.md</code> ; mean macro / min / acc / top-2 vs floor ($\geq$ chance, $\geq \sim 0.5$ desired); per-class F1 sanity (smash / wrist_smash should be the leaders if the wrist-crop carries the right signal)
6	Standard 5-serial run on combo A nosides; A/B against the prevailing baseline (<code>run_20260503_172922</code> plus whatever Capacity Run 2 and aug-landing produce); per-class shifts vs pure BST; project-best gate (mean macro and mean min ws both up)

Critical files to touch (forward inventory)

For when each stage's sub-plan is written:

- `src/bst_refactor/validation_scripts/hit_frame_lookup.py` (Stage 1; extend)
- `src/bst_refactor/validation_scripts/wrist_loss_in_hit_window.py` (Stage 2; new)
- `src/bst_refactor/preparing_data/wrist_crop_spec.py` (Stage 3; new — name TBD)
- `src/bst_refactor/pipeline/wrist_crop_extractor.py` (Stage 4; new)
- `src/bst_refactor/stroke_classification/preparing_data/wrist_crop_dataset.py` (Stage 4; new)
- `src/bst_refactor/stroke_classification/main_on_shuttleaset/x3d_s_train.py` (Stage 5; new)
- `src/bst_refactor/stroke_classification/main_on_shuttleaset/bst_common.py` (Stage 0; extract from existing duplication per `arch_1_directions.md:472-473`)
- `src/bst_refactor/stroke_classification/model/fusion/` (Stage 6; new dir, fusion modules per shortlist)
- `src/bst_refactor/stroke_classification/main_on_shuttleaset/arch1_train.py` (Stage 6; new)
- `src/bst_refactor/stroke_classification/model/bst.py` (Stage 6; possibly extend with optional `x3d_branch` flag, or leave untouched and wrap)
- `src/bst_refactor/stroke_classification/preparing_data/shuttleaset_dataset.py` (Stage 4-6; either extend existing class or build parallel + zip)
- `.env.example` (each stage; new env vars for paths)
- `scratch/architecture_notes/arch_1_directions.md` (each stage; experiment log entries)
- `scratch/architecture_notes/` (each stage; per-stage writeups, names TBD)

What this plan deliberately doesn't do

- Pre-decide the fusion method. That's Stage 6's research arm.
- Pre-decide the training schedule for fusion. Same.
- Re-litigate model choice (X3D-S), input shape ($39 \times \text{stride}=1$), or hit-frame derivation method ($A + B$). Those are settled at `arch_1_directions.md:124-148` and `augmentation_framework.md:790-868`.
- Plan capacity Run 2 or augmentation landing. Both are queued ahead of X3D-S and have their own docs (`transformer_widening_hparam_changes.md`, `augmentation_framework.md`).
- Specify the per-stage code in detail. Each stage is launched as its own task with its own sub-plan; this doc is the gate that says what the sub-plan must answer.